

HTTP Proxy

Popescu Maria-Dariana

Academia Tehnica Militara “Ferdinand I”, Bucuresti, Romania

Strainu Emilian

Academia Tehnica Militara “Ferdinand I”, Bucuresti, Romania

1 Introducere

Acest proiect are ca scop implementarea unui proxy HTTP care intermediază cererile între client și server. Proxy-ul este responsabil pentru gestionarea cererilor HTTP, manipularea acestora și returnarea răspunsurilor. Scopul principal al proxy-ului este de a îmbunătăți performanța, securitatea și flexibilitatea în interacțiunea cu serverele. Proxy-ul implementează un model producer-consumer în care cererile sunt plasate într-o coadă de priorități, iar acestea sunt procesate de mai multe worker threads, care preiau cererile pe baza priorității acestora.

2 Cerințe

2.1 Cerințe Funcționale

- Gestionarea cererilor HTTP: Proxy-ul va intermedia cererile HTTP de tip GET, POST, PUT, și DELETE între client și server.
- Prioritizarea cererilor: Cererile vor fi plasate într-o coadă de priorități și procesate pe baza importanței lor.
- Concurență prin worker threads: Proxy-ul va utiliza mai multe worker threads pentru a procesa cererile simultan.
- Editarea cererilor HTTP: Utilizatorul va putea modifica cererile HTTP (metodă, URL, antete, corp).

- Salvarea cererilor în fișiere de log: Toate cererile și răspunsurile vor fi salvate pentru monitorizare și depanare.
- Securitate: Proxy-ul va proteja serverele backend de atacuri și va gestiona cererile neautorizate.

2.2 Cerințe Nefuncționale

- Performanță bună, cu timpi de răspuns reduși.
- Scalabilitate pentru gestionarea unui număr mare de cereri simultane.
- Securitate împotriva atacurilor, inclusiv protecție la Denial of Service (DoS).

3 Arhitectura și Designul Sistemului

Arhitectura proiectului include mai multe componente care colaborează pentru a intermedia cererile HTTP între client și server. Mai jos este o descriere detaliată a fiecărei componente și a interacțiunii dintre acestea:

3.1 Client

- **Rol:** Clientul trimite cererile HTTP către proxy pentru a fi procesate.
- **Funcționalitate:** Utilizatorul folosește clientul pentru a trimite

cereri către server, iar proxy-ul interceptează aceste cereri și le trimite mai departe.

3.2 Proxy

- **Rol:** Proxy-ul interceptează cererile HTTP și le procesează înainte de a le trimite la server.
- **Funcționalitate:**
 - **Coadă de priorități:** Cererile sunt plasate într-o coadă de priorități pentru a fi procesate într-o ordine determinată de importanța lor.
 - **Worker Threads:** Proxy-ul folosește mai multe worker threads pentru a procesa cererile concurrent.
 - **Editarea cererilor:** Proxy-ul permite modificarea cererilor HTTP (de exemplu, schimbarea metodei, URL-ului, antetelor).
 - **Blocarea cererilor:** Cererile pot fi blocate și trimise cu răspunsuri 403 (Forbidden) dacă sunt considerate nesigure sau nevalide.

3.3 Server

- **Rol:** Serverul primește cererile procesate de proxy și trimite răspunsurile înapoi către proxy.
- **Funcționalitate:** Serverul procesează cererile și returnează răspunsurile prin intermediul proxy-ului către client.

3.4 Fluxul Cererii

1. Clientul trimite cererea HTTP către Proxy.
2. Proxy-ul procesează cererea folosind coada de priorități și worker threads.

Dacă cererea necesită modificări (de exemplu, antete sau URL), aceasta este editată.

3. Proxy-ul trimite cererea procesată către Server.
4. Serverul răspunde la cererea procesată și trimite răspunsul înapoi către Proxy.
5. Proxy-ul returnează răspunsul la Client.

4 Implementare

În această secțiune, vom discuta despre principalele funcționalități implementate în proxy-ul nostru HTTP și despre componentele care au fost dezvoltate pentru a asigura procesarea corectă a cererilor HTTP.

4.1 Coadă de Priorități

Una dintre funcționalitățile esențiale ale proxy-ului este gestionarea cererilor HTTP într-o coadă de priorități. Această coadă permite procesarea cererilor în ordinea importanței lor și permite proxy-ului să prioritizeze cererile mai importante (de exemplu, cererile de administrare sau cele care conțin antete de autentificare). Cererile care așteaptă prea mult în coadă au prioritatea crescută (aging) pentru a preveni starvation-ul cererilor.

4.2 Worker Threads

Pentru a procesa cererile în mod concurrent, proxy-ul folosește worker threads. Acestea sunt fire de execuție care preiau cererile din coadă și le procesează simultan, îmbunătățind astfel performanța aplicației. Fiecare worker thread este responsabil pentru preluarea cererii, verificarea securității cererii, editarea cererii (dacă este necesar),

și trimiterea acesteia la server. De asemenea, fiecare worker thread este sincronizat prin mutex-uri pentru a preveni conflictele în utilizarea resurselor comune (de exemplu, consola).

4.3 Funcții Cheie

În această secțiune, vom detalia funcțiile principale care asigură funcționarea corectă a proxy-ului:

1. `forward_request()`:

- **Descriere:** Această funcție este responsabilă pentru trimiterea cererii HTTP către serverul de backend și returnarea răspunsului către client.
- **Rol:** Proxy-ul primește cererea, o transmite serverului, și trimite înapoi răspunsul clientului.

2. `block_request()`:

- **Descriere:** Utilizatorul are posibilitatea de a bloca trimiterea cererii către server.
- **Rol:** Această funcție previne procesarea cererilor periculoase.

3. `save_request()`:

- **Descriere:** Funcția `save_request()` salvează toate cererile și răspunsurile într-un fișier de log pentru monitorizare și analiză.
- **Rol:** Asigură un registru complet al cererilor și răspunsurilor procesate de proxy.

4. `edit_request()`:

- **Descriere:** Permite utilizatorului să modifice cererile HTTP înainte ca acestea să ajungă la server. Aceasta include schimbarea metodei cererii (GET, POST, PUT, DELETE), a URL-ului sau a antetelor.

- **Rol:** Oferă flexibilitate în procesarea cererilor și permite adăugarea de modificări personalizate.

4.4 Logarea Cererilor și Răspunsurilor

Un alt aspect important al implementării este logarea cererilor și răspunsurilor într-un fișier dedicat. Funcția `save_request()` este folosită pentru a salva detalii despre fiecare cerere procesată și răspunsul returnat de server. Acest lucru ajută la:

- Monitorizarea traficului HTTP.
- Depanarea și analiza cererilor și răspunsurilor.
- Auditorii de securitate pentru a verifica cererile care sunt procesate.

5 Testare și Validare

5.1 Testarea Funcționalităților

Am efectuat teste pentru fiecare funcție majoră a proxy-ului, inclusiv:

- Testarea cererilor GET, POST, PUT, DELETE.
- Testarea prioritizării cererilor.
- Testarea modificării cererilor.
- Testarea salvării cererilor.

5.2 Testarea Performanței

Am măsurat timpul de răspuns pentru diferite tipuri de cereri și am verificat că proxy-ul poate gestiona un număr mare de cereri simultane fără să își piardă performanța.

5.3 Testarea Concurenței

Am testat corectitudinea cozii de priorități și a worker threads. Când mai mulți worker-i procesau cereri, sistemul funcționa corect, iar cererile vechi primeau o prioritate mai mare datorită aging-ului.

6 Concluzii și Posibile Îmbunătățiri

6.1 Realizări

Am implementat un proxy HTTP funcțional, care interceptează cererile, le procesează în paralel folosind worker threads și prioritizează cererile în funcție de importanța acestora. De asemenea, am adăugat funcționalități de editare a cererilor și logare a acestora într-un fișier de log.

6.2 Îmbunătățiri Posibile

- Adăugarea unui sistem de load balancing pentru mai multe instanțe de proxy.
- Implementarea unor algoritmi mai avansați de criptare pentru protecția datelor.
- Optimizarea performanței prin reducerea latenței în procesarea cererilor.

7 Bibliografie

- R. T. Fielding, "REST APIs must be hypertext-driven," IEEE Internet Computing, 2008.
- M. Kerrisk, The Linux Programming Interface: A Linux and UNIX System Programming Handbook, No Starch Press.